What’s the difference between a logical DFD and a physical DFD?

A logical DFD focuses on the business and business activities, while a physical DFD looks at how a system is implemented. So while any data flow diagram maps out the flow of information for a process or system, the logical diagram provides the “what” and the physical provides the “how.” They are two different perspectives on the same data flow, each designed to visualize and improve the system. The logical DFD describes the business events that take place and the data required for each event. It provides a solid basis for the physical DFD, which depicts how the data system will work, such as the hardware, software, paper files and people involved. In tandem, the logical and physical can fully visualize the current state and model the new state to be considered and then implemented.

Purpose and benefits of each

By starting with a **current logical DFD**, you map the flow of business actions as they exist, which can highlight any shortcomings or inefficiencies. Or, you may already know the type of functionality you’re seeking to add, and the current logical DFD will help to reveal process steps that may need to be dropped or changed. As with any diagram, the logical DFD should be detailed enough to be actionable. Depending on its scope, the current logical DFD may take time to produce and seem tedious, but the time may be well spent.

Another benefit of logical DFDs is that they tend to be more easily understandable to non-technical people. They will likely make sense to the people working in the business activities. They will serve as a good tool for collaborating and communicating about better information and functioning, without concern for the “how” yet. They will serve as a bridge from business needs to technical requirements. The discipline of mapping out the current logical flow will help everyone involved to gain a deeper understanding and reveal mistaken assumptions, misunderstandings or shortcomings. Doing logical models reduces the risk of missing business requirements that otherwise would arise belatedly in the process, causing delays and rework.

Then, with a solid understanding of the current business activities, you can model a better way with a **new state logical DFD**, showing new features and functioning based on what the business analysis has revealed. This new logical DFD models what data flows are necessary to create the better functioning, no matter what the technical solution or how the system will be implemented.
After the new logical DFD is drawn, it in turn can be used to figure out the best method to implement the business activities in an upgraded system. This becomes the basis for the **new physical DFD**, depicting that physical implementation of devices, software, files and people to enable the business processes. In this sense, the physical DFD becomes the method of giving the business what it needs. It’s the “how” fueling the “what.” The physical DFD then provides the basis of an implementation plan to provide the new software, hardware, people or other physical pieces needed to run the business process.

An example of logical vs. physical data flow analysis

Let's say your HR department has an outdated approach and system for tracking job applicants. Rather than dive straight into reviewing new software, you start by mapping out the current logical data flow. You detail the business activities that take place, such as actions taken to write a job posting, advertise it, enter applicants into the company's files or database, alert hiring managers, update the files, track process stages, alert candidates and so on. All of this is from the perspective of the business activities, not the technology or other parts of "how." It lays out the current data flow, and it provides the basis for communicating and collaborating on better functionality to accomplish the necessary business actions to sort through job candidates. You then map out a potential new logical flow. For example, it might provide timely alerts to hiring managers, keeping them better apprised. It might allow them easier access to resumes and comparison of finalist's qualifications. This new logical DFD is the basis for discussion of how to best implement better functioning in terms of software, hardware, filing systems and employees, all of which can be visualized in a physical DFD. This can be used to assess software solutions and other implementation pieces and to see which best meets business needs. For example, you could show how different software platforms would vary in different versions of the physical DFD, helping to reveal the best solution.

Contrasting elements of logical vs. physical DFDs

Data flow diagrams are composed of four elements: external entities, processes, data stores and data flows. But the elements represent different perspectives in logical DFDs than in physical DFDs.

For example, in logical DFDs, the processes are business activities; in physical DFDs, the processes are software programs, manual procedures or other ways information is processed. In logical DFDs, the data stores are collections of information, regardless of how they're stored; in physical DFDs, data stores are databases, computer files and paper files.

Entity Relationship Diagram

An entity relationship model, also called an entity-relationship (ER) diagram, is a graphical representation of entities (which will become your tables) and their relationships to each other.

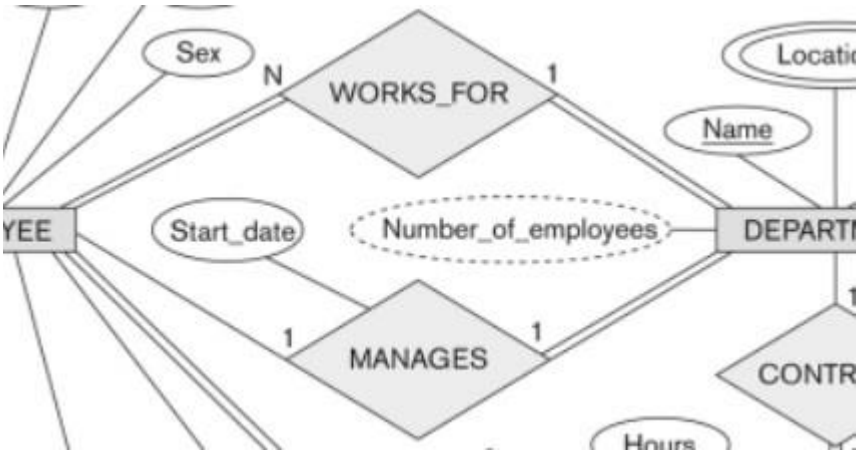
Database modeling *is the process of creating a data model.*

Entity

A simple rectangular block represents a table.

Relationship

It's sketched using the diamond shape.



Relationship

You can sketch the type of the relationship, whether one-to-many using "1-M" , or many-to-many using "M-N", or one-to-one using "1–1".

The lines connecting an entity with a relationship whether single or double line refer to another constrain (we talked about earlier) called "Existence Dependency Constraint" (also called "Participation Constraint").

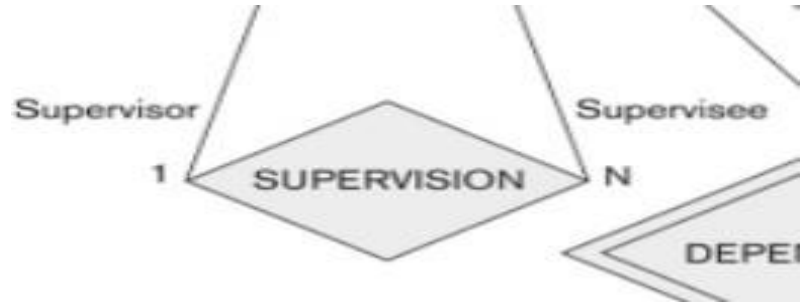
So, for example, if an employee **must** work for a department, this sketched by double lines, and called "total or mandatory participation". And if an employee **may or may not** manage a department, this is sketched by a single line and called "partial or

optional participation”.

A relationship itself can have attributes. In which table we will include these attributes will be discussed in [Database Mapping]() in the next tutorial.

— Recursive Relationship

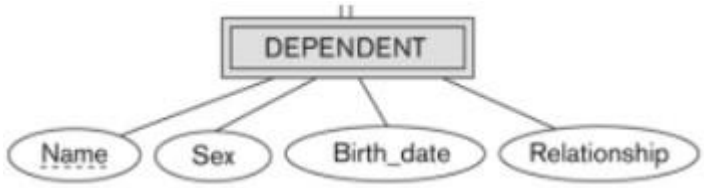
It’s a relationship with the the same entity. For example, an employee may act as a *supervisor* for one or more employees, while an employee may be *supervised by* an employee (it’s one-to-many relationship).



Recursive Relationship

Weak Entity

A weak entity is simply an entity where it’s existence depends on another entity. You can’t logically have dependent(son, daughter, ..etc.) with the absence of the employee table. It’s sketched same as a normal entity but wit double lines.



Weak Entity

— Partial Key

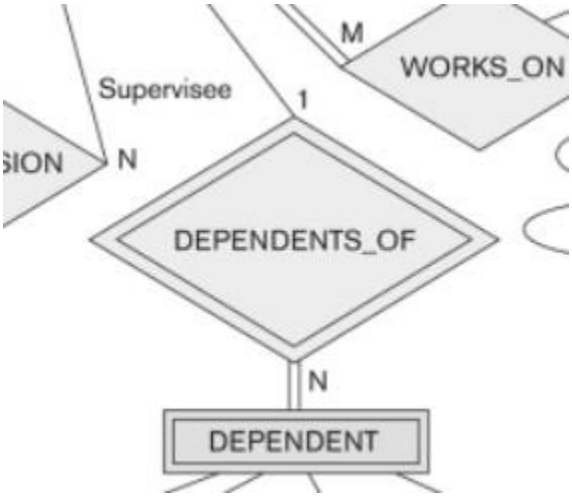
A weak entity has what's called a "partial key". It's one or more attributes that uniquely identify a weak entity for a given owner entity. In our example, the dependent *name* is unique for every employee.

It's sketched the same as a normal attribute, but, with dashed underline.

It's called partial because it can't be a primary key on it's own, it needs another column, which is the foreign key of the owner entity. This will be discussed in [Database Mapping]() in the next tutorial.

— Weak Entity Relationship

Also whenever there is a relationship between a weak entity and another entity, it's sketched same as the one above, but with double lines.



Weak Entity Relationship

Attribute

An attribute or a column (simple) represents a piece of data in the table, like an address, salary, and date.

Attribute

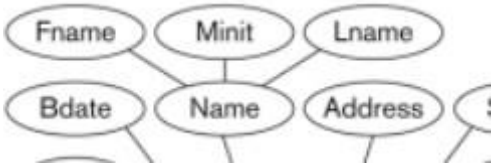
Primary Key

It's sketched the same as a normal attribute, but, with underline.

Primary Key

Composite Attribute

A value that composed of some other values, for example, you may have name that's composed of (*first name, middle name,* and *last name*).



Composite Attribute

We don't define one column for name (if it's [composite](#)), instead we split it up into simple separate columns; first name, last name.

One reason is because you may do something different with the last name(or first name), and you want to get to it by itself. This will be discussed in [Database Mapping]() in the next tutorial.

Multi-valued Attribute

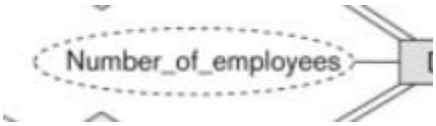
Multiple values for a single column, for example the locations of a department may have more than one value at the same time.

Multi-Valued Attribute

Multi-valued attributes will be extracted in another table. This will be discussed later in [Database Mapping]().

Derived Attribute

When you have a column where it's value could be calculated from another column. Thus, there is no need to insert it's value every time. For example, we can know the number of employees work on a specific department by counting the number of rows.

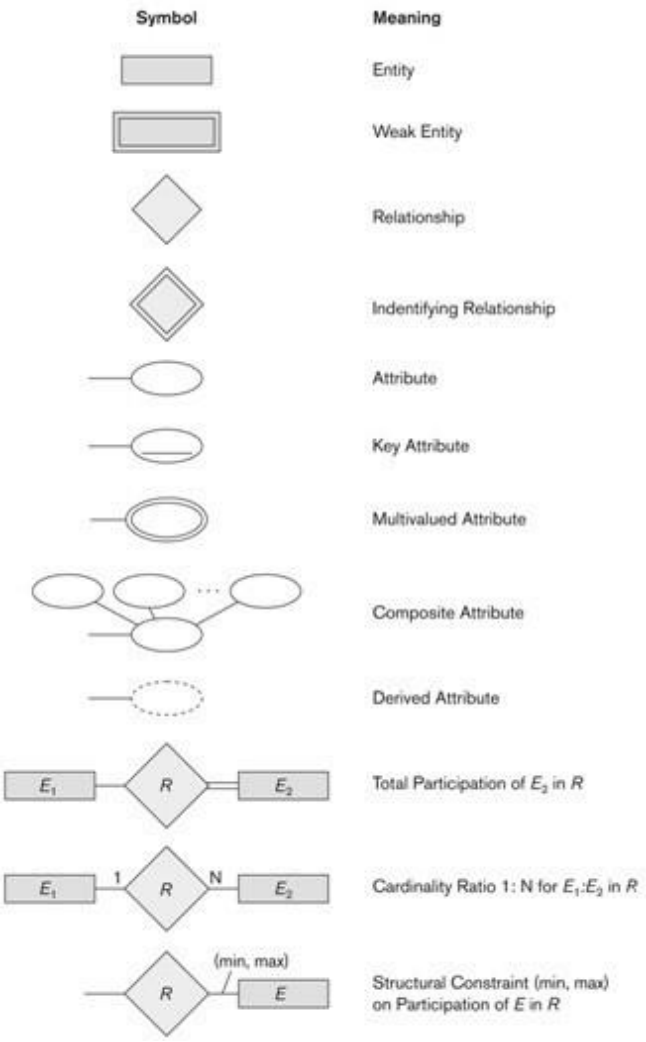


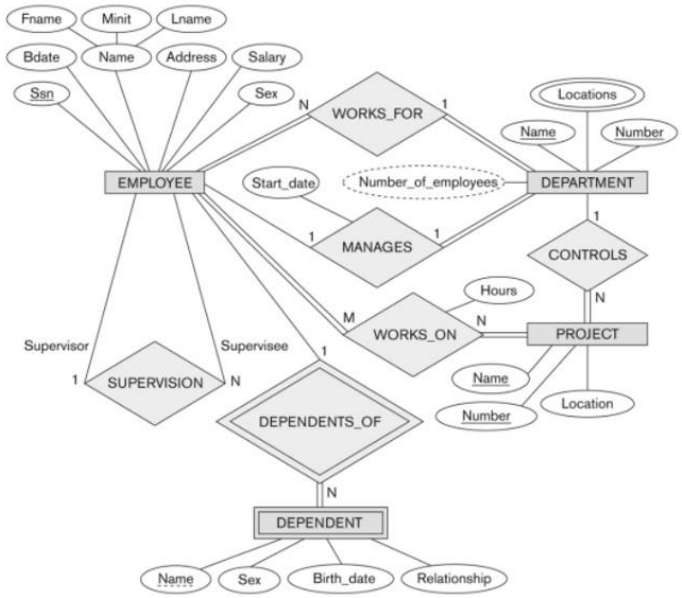
Derived Attribute

This will be done automatically by the DBMS, and it will lead to having a consistent values rather than accidentally inserting a wrong value.

Symbols Summary

Here is a summary for all the symbols in the ERD.




Fname
Minit
Lname
Bdate
Name
Address
Salary
Ssn
Sex
WORKS_FOR
N
1
Locations
Name
Number
EMPLOYEE
Start_date
Number_of_employees
DEPARTMENT
1
MANAGES
1
1
CONTROLS
N
Hours
M
WORKS_ON
N
PROJECT
Supervisor
Supervisee
1
1
SUPERVISION
N
Name
Location
Number
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
Birth_date
Relationship